**框 1. 最佳实践摘要**

1. 为人编写程序，而不是为计算机编写程序。

(a) 程序不应要求读者同时记住过多的事实。

(b) 命名应一致、有区分度且有意义。

(c) 保持代码风格和格式一致。

2. 让计算机完成工作。

(a) 让计算机执行重复任务。

(b) 将最近使用的命令保存到文件中，以便复用。

(c) 使用构建工具自动化工作流。

3. 逐步进行修改。

(a) 小步推进，频繁获取反馈并及时调整方向。

(b) 使用版本控制系统。

(c) 将所有手工创建的内容都纳入版本控制。

4. 不要重复自己或他人已做过的工作。

(a) 每项数据在系统中都必须有唯一的权威表示。

(b) 将代码模块化，而不是复制粘贴。

(c) 复用代码，而不是重新编写。

5. 为可能出现的错误做好准备。

(a) 在程序中添加断言，检查其运行情况。

(b) 使用现成的单元测试库。

(c) 将缺陷转化为测试用例。

(d) 使用符号调试器。

6. 软件正确运行后再进行优化。

(a) 使用性能分析工具识别瓶颈。

(b) 使用尽可能高级的语言编写代码。

7. 记录设计与目的，而不是实现细节。

(a) 记录接口和设计理由，而不是具体实现。

(b) 优先重构代码，而不是解释代码如何工作。

(c) 将软件文档嵌入该软件中。

8. 开展协作。

(a) 在合并代码之前进行审查。

(b) 在帮助新人熟悉工作和解决特别棘手的问题时，采用结对编程。

(c) 使用问题跟踪工具。

## 为人编写程序，而不是为计算机

科学家编写的软件代码既要正确执行，也要便于其他程序员阅读和理解，尤其是未来的自己。如果软件难以阅读和理解，就更难判断它是否真正完成了预期的工作。为了提高工作效率，软件开发者必须考虑人类认知的几个特点：人的工作记忆容量有限，模式识别能力十分敏锐，而持续注意的时间很短[33–37]。

首先，程序不应要求读者一次记住过多事实(1a)。人的工作记忆一次只能容纳少量项目，每个项目可以是单个事实，也可以是由多个事实组成的“组块”[33,34]。因此，程序应限制完成一项任务时需要记住的项目总数。主要方法是把程序拆分为易于理解的函数，让每个函数只完成一项容易理解的任务。这样就能让程序的各个部分更容易理解，正如用章节和段落组织科学论文可以提高其可读性一样。

其次，科学家应采用一致、有区分度且有意义的命名(1b)。例如，使用 a 和 foo 这类缺少描述性的名字，或 results 和 results2 这类非常相似的名字，都容易造成混淆。

第三，科学家应保持代码风格和格式一致(1c)。如果一篇科学论文的不同部分采用不同的格式和大小写规则，就会更难阅读。同样，如果程序各部分的缩进不一致，或混用 CamelCaseNaming 与 pothole_case_naming，阅读代码就会花费更多时间，读者也更容易犯错[35,36]。

## 让计算机完成工作

科学研究经常涉及重复的计算任务，例如以相同方式处理大量数据文件，或在每次向现有分析添加新数据时重新生成图表。计算机正是为执行这些重复任务而发明的，但即使在今天，许多科学家仍会一遍遍输入相同命令，或反复点击相同按钮[17]。这不仅浪费时间，而且即使最细心的研究者，也迟早会在重复操作中分心并出错。

因此，科学家应让计算机执行重复任务(2a)，并将最近使用的命令保存到文件中以便复用(2b)。例如，大多数命令行工具都有“历史记录”选项，可以显示并重新执行最近使用的命令，只需对文件名或参数作少量修改。这也常被认为是命令行界面仍然流行的原因之一[38,39]：“再执行一次”既节省时间，也减少错误。

包含交互式系统命令的文件通常称为脚本，其实脚本与程序并没有区别。当需要以相同方式或组合方式反复使用这些脚本时，可以使用工作流管理工具。典型例子是编译和链接用 Fortran、C++、Java 和 C#等语言编写的程序[40]。这项工作中使用最广泛的工具可能是 Make（http://www.gnu.org/software/make），不过如今也有许多替代工具[41]。这些工具都允许用户表达文件之间的依赖关系：如果 A 或 B 发生变化，就需要使用一组特定命令更新 C。这些工具也已成功应用于科学工作流[42]。

为了避免手工重复执行命令带来的错误和低效率，我们建议科学家使用构建工具自动化工作流(2c)。例如，明确中间数据文件、最终结果以及生成它们的程序之间的依赖关系，这样只需一条命令，就能重新生成所有需要更新的内容。